

azi 4:15

I'd start by treating the SDLC not as a document, but as an executable protocol: states, artifacts, permissions, gates, and recoverable transitions. I'm grounding this in the orchestrator direction you were already exploring, especially your concern about keeping one controlling session and restricting harness permissions.

Thought for 43s >

I would **not start by writing prompts**.

I would start by designing an **SDLC runtime** in which prompts are merely one execution mechanism. Otherwise, you risk creating a majestic folder full of Markdown that behaves like a very confident intern.

The central idea would be:

The SDLC is an executable protocol that transforms a user intention into

verified evidence and deliverable artifacts.

The LLM may perform actions itself, delegate them to another harness, call MCP tools, request human decisions, or stop when policy forbids proceeding.

1. Define the SDLC kernel

The kernel should be independent of:

- programming language;
- repository host;
- LLM provider;
- agent harness;
- deployment platform;
- development methodology;
- project size.

Its job is to manage six things:

```
graph TD;
  Intent --> Obligations;
  Obligations --> Actions;
  Actions --> Artifacts;
  Artifacts --> Evidence;
  Evidence --> Outcome;
```

I would call this layer something like:

- **Development Execution Protocol**
- **SDLC Kernel**
- **Development Control Plane**
- **Engineering Execution Runtime**

“SDLC prompts” would then be one adapter sitting on top of this runtime.

2. Model work as obligations, not phases

Traditional SDLCs prescribe phases:

```
Requirements → Design → Development → Testing → Deployment
```

That is too rigid for an LLM. Fixing a typo does not require a five-page architecture review. Replacing an authentication system absolutely does.

Instead, every task should produce a set of **obligations**.

For example, a bug fix might generate:

obligations:

- understand_current_behavior
- reproduce_defect
- identify_root_cause
- define_expected_behavior
- implement_change

YAML

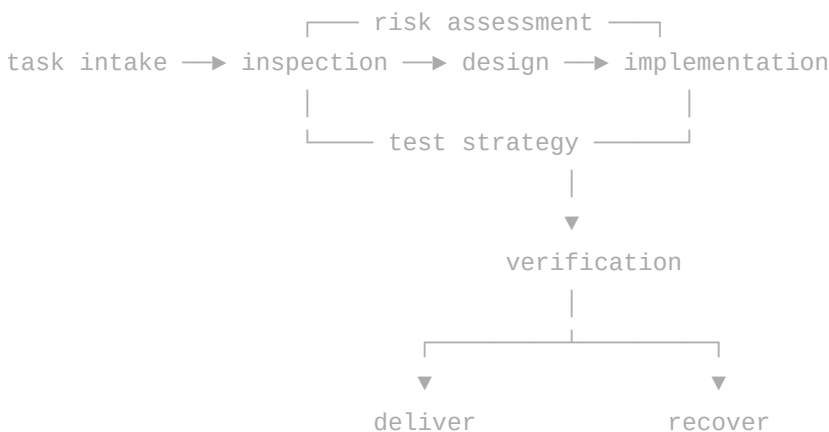
A database migration might generate:

obligations:

- inspect_current_schema
- define_target_schema
- assess_data_loss_risk
- produce_forward_migration
- produce_rollback_strategy

YAML

The lifecycle becomes a **dynamic graph** generated from the obligations:



Phases still exist, but they are selected because the task requires them—not because the sacred waterfall tablet said so.

3. Create a canonical task contract

Every user request should first be transformed into a structured task contract.

```
task:
  id: TASK-1042
  intent: fix_defect

objective:
  statement: >
    Prevent duplicate invoices when a payment webhook is delivered
    more than once.

success_conditions:
  - repeated webhook delivery creates only one invoice
  - valid first-time webhook delivery still creates an invoice
  - existing invoices are not modified

constraints:
  - preserve public API behaviour
  - no database downtime
  - PostgreSQL 16
  - must remain backwards compatible

context:
  repository: billing-service
  affected_area:
    - webhook handler
    - invoice persistence

risk:
  initial_level: high
  reasons:
    - financial data
    - production persistence
    - webhook concurrency

permissions:
  repository_read: true
  repository_write: scoped
  command_execution: sandboxed
  network_access: false
  deployment: forbidden
```

```
budgets:
  max_cost: 2.00
  max_duration_seconds: 1800
  max_context_tokens: 50000

unknowns:

YAML
```

This contract becomes the source of truth. The initial user message does not remain the sole specification forever.

The orchestrator may update the contract, but every update should be recorded as a decision or newly discovered fact.

4. Introduce a small universal ontology

I would make these first-class objects:

Object	Purpose
Task	The user’s requested outcome
Obligation	Something that must be satisfied
Action	A bounded unit of work
Artifact	A persistent output
Claim	Something asserted to be true
Evidence	Proof supporting a claim
Decision	A selected option with rationale
Risk	A possible undesirable outcome
Gate	A policy check before proceeding
Capability	Something an actor or tool can do
Actor	Human, LLM, harness, MCP, CI system

Object**Purpose**

Permission

Authority to perform an action

Event

Immutable record of what happened

The important innovation is the relationship between **claims and evidence**.

An LLM should not say:

```
The implementation works.
```

It should produce:

```
claim:
```

```
  id: CLAIM-17
```

```
  statement: Duplicate webhook delivery creates only one invoice.
```

```
evidence:
```

```
  - type: automated_test
```

```
    artifact: test-idempotent-webhook
```

```
    result: passed
```

```
  - type: database_assertion
```

```
    query: count invoices by payment_id
```

```
YAML
```

That gives you a generic definition of “done”:

A task is complete when all required obligations are satisfied, all completion claims have acceptable evidence, and all applicable gates pass.

5. Separate commands from prompts

Commands should express **lifecycle intentions**, not contain giant hidden workflows.

For example:

```
/sdlc start
```

```
/sdlc inspect
```

```
/sdlc define  
/sdlc plan  
/sdlc execute  
/sdlc verify  
/sdlc review  
/sdlc deliver  
/sdlc recover  
/sdlc status  
/sdlc explain
```

But internally, these commands should invoke protocol operations:

```
task.create  
task.classify  
obligations.derive  
context.acquire  
plan.construct  
action.dispatch  
evidence.record  
gate.evaluate  
task.transition
```

That distinction matters.

`/sdlc verify` is a user-facing command.

`gate.evaluate` is an internal runtime operation.

A GitHub issue comment, CLI command, Slack message, HTTP API, or autonomous agent could all invoke the same internal operation.

6. Make prompts compiled artifacts

Do not manually maintain one gigantic prompt per agent.

Instead, generate prompts from several components:

```
Prompt =  
  Role Policy  
  + Current Task Contract  
  + Assigned Action  
  + Available Capabilities  
  + Permission Boundaries  
  + Relevant Context
```

- + Required Output Schema
- + Completion Conditions

For example, the implementation prompt might be compiled from:

```
actor:
  role: implementer
  model: qwen-coder
  harness: opencode

action:
  id: ACTION-31
  type: modify_code
  objective: add transaction-safe webhook idempotency

allowed_capabilities:
  - repository.read
  - repository.edit
  - tests.run

forbidden_capabilities:
  - git.push
  - deployment.execute
  - network.access
  - secrets.read

required_outputs:
  - patch
  - changed-files manifest
  - implementation claims
  - unresolved issues

completion_conditions:
  - targeted tests pass

YAML
```

Then the compiler produces the actual harness-specific prompt.

This also means OpenCode, Pi, Codex, Claude Code, a local model, or a custom agent can receive different prompt formats while following the same protocol.

7. Treat MCPs and tools as capability providers

Do not design around named tools such as “GitHub MCP” or “Playwright MCP.”
Define abstract capabilities first.

```
capability: repository.search
```

```
inputs:  
  query:  
    type: string
```

```
outputs:  
  matches:  
    type: array
```

```
properties:  
  mutating: false  
  deterministic: mostly
```

YAML

A tool registry can map capabilities to providers:

```
repository.search:  
  providers:  
    - sourcegraph-mcp  
    - local-code-index  
    - ripgrep-adapter
```

```
tests.run:  
  providers:  
    - sandbox-shell  
    - github-actions  
    - container-runner
```

```
issue.read:  
  providers:  
    - github-mcp
```

YAML

The orchestrator selects a provider based on:

- availability;
- permissions;

- cost;
- latency;
- trust;
- environment;
- task risk.

That supports your earlier **cheapest-sufficient-agent** idea at the tool level as well as the model level.

8. Build permissions into the protocol

This should not be an afterthought.

The orchestrator should issue a temporary **capability lease** to each actor.

```
lease:
  actor: opencode-worker-7
  task: TASK-1042

workspace:
  type: git_worktree
  path: /workspaces/TASK-1042

capabilities:
  repository.read:
    scope: entire_worktree

  repository.edit:
    scope:
      - src/webhooks/**
      - tests/webhooks/**

  command.execute:
    allow:
      - npm test
      - npm run lint
      - npm run typecheck

  network.access:
    allowed: false

  git.commit:
    allowed: false

  git.push:
```

```
allowed: false
```

YAML

The harness should not receive general OS access. It receives:

- an isolated worktree or container;
- mounted files;
- an allowlisted command runner;
- scoped credentials;
- capability-specific tool endpoints.

Your orchestrator remains the authority. OpenCode or Pi becomes an execution worker, not the owner of the machine and not the owner of the lifecycle.

9. Make every execution event-sourced

Every meaningful operation should emit an event:

```
{
  "event": "action.completed",
  "task_id": "TASK-1042",
  "action_id": "ACTION-31",
  "actor": "opencode-worker-7",
  "timestamp": "2026-07-15T14:34:10Z",
  "outputs": [
    "artifact://patches/ACTION-31.diff",
    "artifact://reports/ACTION-31-tests.json"
  ],
  "claims": ["CLAIM-17"],
```

JSON

This gives you:

- resumability;
- auditability;
- context reconstruction;
- cost tracking;
- failure recovery;
- multi-agent coordination;
- explainability.

A new orchestrator session should be able to reconstruct the task from the event log and artifact store rather than needing the original 80,000-token conversation.

10. Use gates as executable policies

A gate should be deterministic wherever possible.

```
gate:
  id: release-readiness

  applies_when:
    task.risk: [medium, high, critical]

  conditions:
    - all_required_tests_pass
    - no_unresolved_critical_risks
    - all_requirements_have_evidence
    - security_scan_passes
    - rollback_artifact_exists
```

YAML

A gate can contain three types of checks:

```
Deterministic
  Tests passed, files exist, coverage threshold met.

Tool-evaluated
  Static analysis, dependency scan, schema validation.

Judgement-based
  Architecture quality, requirement completeness, usability.
```

LLMs may evaluate judgement-based checks, but they should return structured findings and evidence—not merely `APPROVED`.

11. Avoid a single “planner → coder → reviewer” pipeline

Those are useful roles, but they should not define the architecture.

Instead, an action can request capabilities:

```

action:
  type: root_cause_analysis

requires:
  - code_search
  - execution_trace_analysis

```

YAML

The router can decide that:

- a cheap local model searches and summarizes;
- a stronger reasoning model generates hypotheses;
- a sandbox tool reproduces the bug;
- another model reviews the conclusion;
- the orchestrator decides whether enough evidence exists.

Roles become temporary **execution profiles**, not permanent little digital employees with colourful job titles and a tendency to hold meetings.

12. Start with task archetypes

“Generic development” is too broad to test directly. I would begin with three archetypes:

Bug fix

```

Intake
→ Reproduce
→ Diagnose
→ Define expected behaviour
→ Implement
→ Regression test
→ Review
→ Deliver

```

Feature

```

Intake
→ Clarify outcome
→ Derive requirements
→ Inspect existing system
→ Design

```


- Decompose
- Implement
- Verify
- Deliver

Refactor

- Intake
- Define preserved behaviour
 - Establish baseline evidence
 - Identify change boundary
 - Refactor
 - Compare behaviour
 - Review maintainability
 - Deliver

These three are different enough to expose whether your protocol is genuinely dynamic.

Later, add:

- migration;
- dependency update;
- security remediation;
- performance investigation;
- incident response;
- research spike;
- documentation change;
- greenfield project;
- release preparation.

13. Build the first version as a control plane

A reasonable first repository might look like:

```
sdlc-runtime/  
├─ schemas/  
│   ├─ task.schema.json  
│   ├─ obligation.schema.json  
│   ├─ action.schema.json  
│   ├─ artifact.schema.json  
│   ├─ evidence.schema.json  
│   └─ event.schema.json
```

```
|
|— policies/
|   |— permissions.yaml
|   |— risk.yaml
|   |— gates.yaml
|   └─ routing.yaml
|
|— archetypes/
|   |— bug-fix.yaml
|   |— feature.yaml
|   └─ refactor.yaml
|
|— capabilities/
|   |— repository.yaml
|   |— workspace.yaml
|   |— testing.yaml
|   └─ git.yaml
|
|— prompts/
|   |— fragments/
|   └─ output-contracts/
|
|— adapters/
|   |— opencode/
|   |— pi/
|   |— mcp/
|   └─ local-shell/
|
|— runtime/
|   |— orchestrator
|   |— planner
|   |— prompt-compiler
|   |— capability-router
|   |— policy-engine
|   └─ event-store
|
└─ cli/
    └─ sdlc
```

14. The first runnable vertical slice

I would implement this exact scenario first:

User: Fix issue #123.

The runtime should:

1. Read the issue.

2. Create a task contract.
3. Inspect the repository without modifying it.
4. Classify the task as a bug fix.
5. Derive obligations.
6. Generate an execution graph.
7. Create an isolated worktree.
8. Give a worker read access.
9. Ask it to reproduce and diagnose the defect.
10. Store its hypotheses and evidence.
11. Authorize edits only after diagnosis.
12. Run tests through a controlled runner.
13. Ask a separate reviewer to challenge the claims.
14. Evaluate completion gates.
15. Produce a patch or draft pull request.
16. Return a concise explanation with evidence.

The success criterion for version one is not:

The agent fixed the bug.

It is:

The runtime can explain:

- what it believed the task was;
- what obligations it derived;
- which actors performed which actions;
- what permissions they received;
- what changed;
- what claims were made;
- what evidence supports those claims;
- which gates passed;
- what remains uncertain.

15. The deepest architectural rule

I would write this at the top of the project constitution:

LLM output is a proposal until converted into an artifact, observation, decision, or evidence record accepted by the runtime.

That one rule prevents a great deal of agent theatre.

The LLM can say it ran the tests. The runtime trusts the test-runner event.

The LLM can say a requirement is satisfied. The runtime asks for the corresponding claim and evidence.

The LLM can say the change is safe. The runtime checks risk policy, changed files, tests, and reviewer findings.

My recommended first design deliverable

Before implementing prompts, write these five schemas:

```
Task
Obligation
Action
Evidence
Event
```

Then define:

```
derive obligations
construct action graph
lease capabilities
execute action
record evidence
evaluate gate
transition task
```

Once those operations exist, prompts, commands, MCPs, agents, and harnesses have somewhere coherent to plug in.

Without that kernel, you are building an elaborate prompt library.

With it, you are building an operating system for software development.

     ...  Surse

Holy mother! I don't understand even half of what you are talking about. You're not suggesting a set of prompts. You're suggesting an entire framework here.

Thought for 4s >

Yes. You are absolutely right. I went from “let’s write some useful prompts” to “let’s build Kubernetes for software development.” Slight escalation.

What you actually need first is much simpler:

A small set of reusable prompts that guide an LLM through a development task without letting it immediately start vomiting code.

I would start with **one orchestrator prompt and five command prompts**.

The minimal SDLC

```
/intake
/explore
/plan
/implement
/verify
/finish
```

The LLM moves through these commands depending on the task.

1. /intake

Purpose: understand what the user is actually asking.

You are in the Intake stage of a software development task.

Your goal is to understand the requested outcome before proposing or making changes.

Identify:

- the user's objective;
- the type of task: feature, bug, refactor, investigation, documentation, or other;
- known constraints;
- expected behaviour;
- missing information;
- likely risks.

Do not write code.

Do not create an implementation plan yet.

Do not invent requirements.

Return:

Objective

Task type

Known requirements

Constraints

Unknowns

Risks

Recommended next step

This converts:

Add login with Google

into something more usable than the LLM yelling, "Certainly!" and installing twelve authentication libraries.

2. /explore

Purpose: understand the existing project.

You are in the Exploration stage.

Inspect the available project context before suggesting changes.

Determine:

- how the relevant part of the system currently works;
- which files and components are involved;
- existing conventions and architectural patterns;
- relevant tests;
- dependencies and integrations;
- possible impact on other parts of the system.

Do not modify files.

Do not produce implementation code.

Separate confirmed facts from assumptions.

Return:

Current behaviour

Relevant components

Existing patterns

Tests and validation

Dependencies

Impact areas

Assumptions

Findings

This prompt may use:

- file-search tools;
- repository tools;
- GitHub MCP;
- shell commands;
- code search;
- documentation search.

The prompt defines the job. Tools provide the eyes and hands.

3. /plan

Purpose: decide what should be done.

You are in the Planning stage.

Using the task definition and exploration findings, create a concrete implement

The plan must:

- satisfy the requested outcome;
- follow existing project conventions;
- minimize unnecessary changes;
- identify files likely to change;

- include testing and verification;
- identify risks and unresolved decisions.

Do not implement the changes.

Return:

Proposed approach

Steps

For each step include:

- objective;
- affected files or components;
- expected change;
- validation method.

Risks

Open decisions

Definition of done

The output should be executable enough that either the same LLM or another agent can implement it.

4. /implement

Purpose: carry out the approved plan.

You are in the Implementation stage.

Implement the agreed plan.

Rules:

- make only changes required by the plan;
- preserve existing behaviour unless explicitly instructed otherwise;
- follow project conventions;
- do not silently expand the scope;
- add or update tests where appropriate;
- report deviations from the plan;
- do not claim that something works unless it has been verified.

During implementation, keep track of:

- files changed;
- decisions made;
- deviations;
- unresolved issues.

Return:

Changes made

Files changed

Tests added or updated

Deviations from plan

Remaining concerns

This stage gets access to editing tools.

The earlier stages ideally do not.

That alone gives you a primitive permission model without inventing the Department of Agentic Bureaucracy.

5. /verify

Purpose: check the work independently.

You are in the Verification stage.

Verify whether the implementation satisfies the original task and the agreed pl

Perform all available relevant checks, such as:

- automated tests;
- type checking;
- linting;
- build validation;
- behavioural inspection;
- regression checks;
- security or performance checks where relevant.

Do not fix failures unless explicitly instructed.

Do not treat implementation claims as evidence.

Return:

Checks performed

Results

Requirements satisfied

Failures

Potential regressions

Verdict

Use one verdict:

- PASS
- PASS WITH CONCERNS
- FAIL
- UNABLE TO VERIFY

This can be performed by another LLM or agent to reduce “I reviewed my own homework and awarded myself a gold star.”

6. /finish

Purpose: present the result to the user.

You are in the Completion stage.

Summarize the completed development task for the user.

Include:

- what was requested;
- what was changed;
- important technical decisions;
- verification results;
- remaining limitations or follow-up work.

Be concise.

Do not claim success if verification failed.

Do not hide unresolved concerns.

Return:

Result

Main changes

Verification

Remaining concerns

The orchestrator prompt

This is the piece that decides which prompt should run next.

You are the coordinator of a software development workflow.

The available stages are:

1. intake
2. explore
3. plan
4. implement
5. verify
6. finish

Your job is to select the next appropriate stage.

Rules:

- do not implement before the task is understood;
- explore the existing system before modifying an existing project;
- create a plan before making non-trivial changes;
- verify every implementation;
- return to planning when exploration contradicts the current plan;
- return to implementation when verification finds fixable defects;
- do not declare completion when verification fails;
- skip unnecessary stages for trivial tasks;
- ask the user only for information that cannot be discovered using available t

Return:

NEXT_STAGE: <stage>

REASON: <brief explanation>

REQUIRED_INPUTS:

- <inputs needed by the next stage>

The first version does not require a framework.

It can literally be:

```
User request
  ↓
Orchestrator prompt
  ↓
Selected stage prompt
  ↓
Tool calls
  ↓
Result passed back to orchestrator
```

What I would build first

Start with only these four:

```
/explore
/plan
/implement
/verify
```

Run them manually at first.

Once they produce good results, add the orchestrator that chooses between them. Only after that should MCPs, permissions, parallel agents, persistent state, and the previous monstrosity enter the room.

So yes: **prompts first, framework later—only when the prompts expose a real need for one.**



Retake this idea but with sub-agents and the capacity to start from dumber free agents and escalating the job to more powerful agents when they fail or when orchestrator devices the provided result is not good enough. I'm even willing to

introduce the idea of giving the same LLM type the same job multiple times (even in parallel) and then picking the best solution.

Yes—this is the useful version: a lightweight SDLC driven by an orchestrator, cheap worker agents, repeated attempts, and escalation only when needed. No agentic Ministry of Bureaucracy this time.

Thought for 39s >

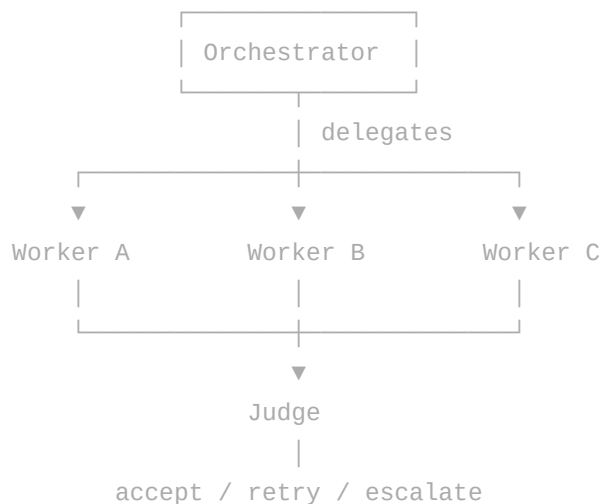
Exactly. The SDLC remains simple:

Understand → Explore → Plan → Implement → Verify → Finish

The innovation is that **each stage may be performed by one or more sub-agents**, starting with cheap/free models. The orchestrator accepts, retries, compares, or escalates their work.

The minimal architecture

You need only three concepts:



The orchestrator does not necessarily perform the development work. It:

- decides which SDLC stage comes next;
- chooses the model tier;
- decides how many attempts to run;
- gives agents tools and context;
- evaluates the results;

- escalates when necessary.

The **judge** can be a separate agent, the orchestrator itself, or preferably deterministic tools when possible.

1. The agent ladder

Define model tiers based on cost and capability, without hard-coding particular providers.

```
tiers:
  tier_0:
    description: free or local models
    use_for:
      - repository search
      - file identification
      - simple summaries
      - trivial edits
      - first attempts

  tier_1:
    description: inexpensive general-purpose models
    use_for:
      - normal planning
      - common bug fixes
      - test generation
      - code review

  tier_2:
    description: powerful reasoning or coding models
    use_for:
      - difficult diagnosis
      - architecture
      - conflicting requirements
      - failed implementations
      - security-sensitive work

  tier_3:
    description: premium specialist or human decision
    use_for:
      - critical-risk changes
      - unresolved ambiguity
```

YAML

The default rule is:

Start with the cheapest model that has a reasonable chance of succeeding.

Not necessarily one free agent, though. Three free attempts may still be cheaper and better than one expensive attempt.

2. The execution strategies

For every SDLC stage, the orchestrator chooses one of four strategies.

Single attempt

Use one agent when the job is simple and objectively verifiable.

Agent → Result → Check → Accept

Example: finding where a function is defined.

Retry with feedback

Run the same agent type again, but include the previous failure.

Attempt 1
↓
Evaluation feedback
↓
Attempt 2

This is useful when the first answer is close but incomplete.

Parallel ensemble

Give the same job to multiple independent agents.

Task ———┐ Agent A – Solution A
 ├ Agent B – Solution B
 └ Agent C – Solution C
 ↓
 Judge

This is ideal for:

- architecture decisions;
- root-cause analysis;
- implementation plans;
- difficult bugs;

- alternative algorithms.

Escalation

Move the task to a stronger model when cheaper attempts cannot meet the acceptance criteria.

```
Tier 0 → retry Tier 0 → parallel Tier 0 → Tier 1 → Tier 2
```

The orchestrator should not escalate merely because an agent says, “This is complicated.” Agents can be dramatic little creatures.

Escalation should be triggered by observable problems.

3. The SDLC stages with sub-agents

/intake

The orchestrator interprets the user request and creates a task description.

Usually, one inexpensive agent is enough. For ambiguous requests, run two interpretations and compare them.

Output:

```
objective: Prevent duplicate invoices from webhook retries
```

```
task_type: bug_fix
```

```
success_conditions:
```

- one invoice per payment
- first webhook still creates an invoice
- duplicate webhook does not modify existing invoice

```
constraints:
```

- preserve public API
- no database downtime

YAML

The orchestrator may ask the user only about unknowns that cannot be discovered from the project.

/explore

Cheap agents are especially useful here because exploration can be split into small jobs.

```
Scout 1: Find relevant source files
Scout 2: Find relevant tests
Scout 3: Find similar patterns elsewhere
Scout 4: Inspect dependencies and configuration
```

These can all be the same model type.

The orchestrator combines their findings:

```
confirmed:
- webhook handler is in src/payments/webhook.ts
- invoices are inserted through InvoiceRepository
- no unique constraint exists on payment_id
```

```
assumptions:
- webhooks may execute concurrently
```

```
conflicts:
```

```
YAML
```

When agents disagree, the orchestrator can:

1. ask another cheap agent to investigate the exact disagreement;
2. use a repository or execution tool to verify it;
3. escalate only if the conflict remains unresolved.

/plan

This stage benefits heavily from multiple candidates.

The orchestrator might request:

```
Planner A: produce the smallest safe change
Planner B: produce the most robust long-term solution
```

Planner C: challenge assumptions and identify failure cases

These could all be three instances of the same free model.

A judge then scores the plans against a rubric:

```
criteria:
  correctness: 30
  requirement_coverage: 25
  project_compatibility: 20
```

YAML

The judge should return something like:

```
candidate_a:
  score: 72
  problems:
    - prevents sequential duplicates but not concurrent duplicates

candidate_b:
  score: 91
  problems:
    - requires a database migration

candidate_c:
  score: 63
  problems:
    - identifies risks but does not provide a complete implementation plan
```

YAML

The orchestrator may also synthesize a final plan from multiple candidates rather than selecting one unchanged.

/implement

The selected plan is given to an implementation agent.

For small changes:

One cheap agent → tests → accept or retry

For difficult changes:

```
Builder A → implementation A
Builder B → implementation B
          ↓
      tests and comparison
```

Running multiple implementations is more expensive, so reserve it for situations where:

- solutions are small enough to compare;
- the bug is difficult;
- the design has several plausible approaches;
- correctness matters more than token cost.

Each implementation agent should work independently. Do not show Builder B the work of Builder A unless you are explicitly running a review or improvement pass.

/verify

Verification should not rely solely on another LLM's opinion.

Use tools first:

```
Tests
Type checker
Linter
Build
Static analysis
Security scanner
Performance benchmark
```

Then use verifier agents for things tools cannot determine easily:

- Does the implementation satisfy the original intent?
- Were requirements overlooked?
- Does the change follow project conventions?
- Are there subtle regressions?
- Is the solution unnecessarily complex?

You can run several verifier agents:

```
Verifier A: requirement compliance
Verifier B: bug and edge-case review
Verifier C: maintainability review
```

If verification fails, the orchestrator sends the implementation back with concrete feedback.

```
Implement → Verify → Repair → Verify
```

After repeated failures, escalate the implementation agent.

/finish

The orchestrator produces the final result from accepted artifacts:

- what was requested;
- what was changed;
- what was verified;
- what remains uncertain.

A cheap model can usually write this summary because it is not making technical decisions anymore.

4. The common worker prompt

You do not need a completely different prompt for every worker. Start with one generic template.

```
You are a sub-agent working on one stage of a software development task.
```

```
STAGE:
{{stage}}
```

```
ASSIGNMENT:
{{assignment}}
```

```
ORIGINAL USER REQUEST:
{{user_request}}
```

```
KNOWN CONTEXT:
{{context}}
```

CONSTRAINTS:

{{constraints}}

AVAILABLE TOOLS:

{{tools}}

You must:

1. Work only on the assigned task.
2. Separate confirmed facts from assumptions.
3. Use available tools when they can verify a claim.
4. Do not expand the task scope.
5. Report incomplete or uncertain findings clearly.
6. Do not claim that commands or tests succeeded unless they were actually run.

Return:

Result

Evidence

Assumptions

Risks or problems

Suggested next action

The same prompt can be used by:

- scouts;
- planners;
- implementers;
- reviewers;
- verifiers.

The assignment changes, while the basic behavioural contract remains consistent.

5. The judge prompt

The judge must compare candidates against the task—not against which answer sounds cleverest.

You are evaluating candidate results produced for an SDLC stage.

ORIGINAL TASK:

{{task}}

STAGE:

{{stage}}

ACCEPTANCE CRITERIA:

{{criteria}}

CANDIDATES:

{{candidate_results}}

Evaluate each candidate independently.

For every candidate:

1. Identify what it accomplished.
2. Check whether its claims are supported by evidence.
3. Identify missing requirements.
4. Identify incorrect assumptions.
5. Identify risks and unnecessary complexity.
6. Score it against each acceptance criterion.

Do not prefer a candidate because it is longer or more confident.

Return:

Candidate evaluations

Ranking

Selected candidate

Elements worth preserving from rejected candidates

Verdict

Use one verdict:

- ACCEPT
- SYNTHESIZE
- RETRY
- ESCALATE
- BLOCKED

For implementation candidates, replace persuasive prose with tool evidence wherever possible. Passing tests beats a beautifully written declaration of success.

6. The orchestrator prompt

You orchestrate a software development lifecycle using sub-agents.

The available stages are:

- intake
- explore
- plan
- implement
- verify
- finish

Agents are available in capability and cost tiers. Always begin with the least expensive tier reasonably capable of the assignment.

For each stage, choose one execution strategy:

- SINGLE: one agent attempt;
- RETRY: rerun after providing evaluation feedback;
- ENSEMBLE: run independent agents in parallel;
- ESCALATE: assign the work to a more capable tier.

Use ENSEMBLE when:

- multiple valid approaches may exist;
- the task is ambiguous;
- independent investigation would improve confidence;
- previous attempts disagree;
- the cost of failure is significant.

Use ESCALATE when:

- repeated attempts fail acceptance criteria;
- agents cannot resolve contradictions;
- verification repeatedly fails;
- the required reasoning exceeds the current tier;
- the task has significant security, financial, privacy, or production risk.

Do not escalate merely because the agent reports low confidence.

First determine whether tools, another attempt, or independent agents can resolve the issue.

Do not accept results based only on self-reported confidence.

For every decision return:

STAGE:

STRATEGY:

AGENT_TIER:

AGENT_COUNT:
 ASSIGNMENTS:
 ACCEPTANCE_CRITERIA:
 REASON:

7. Escalation policy

Keep it understandable rather than constructing an intergalactic decision engine.

```
default_policy:
  first_attempt:
    tier: 0
    agents: 1

  incomplete_result:
    action: retry
    maximum_retries: 1

  ambiguous_or_high_variance_task:
    action: ensemble
    agents: 3

  disagreement_between_agents:
    action: targeted_investigation

  failed_objective_verification:
    action: retry_with_feedback

  repeated_failure:
    action: escalate_one_tier

  critical_risk:
    minimum_tier: 2
```

YAML

A realistic sequence might be:

```
Tier-0 agent fails
↓
Tier-0 retry with feedback fails
↓
Three Tier-0 agents run independently
```


↓
Judge finds no acceptable candidate
↓
One Tier-1 agent receives:
- original task
- failed attempts
- judge findings
- tool evidence

Give the stronger agent the lessons from previous failures, not merely the original request. Otherwise, you are paying more money to step on the same rake with superior reasoning capabilities.

8. Using the same model multiple times properly

Simply calling the same model three times with the identical prompt may produce three cousins of the same answer.

Introduce deliberate diversity:

Agent A: Find the simplest valid solution.
Agent B: Assume Agent A's likely solution is flawed; search for edge cases.
Agent C: Find a solution optimized for maintainability.

For debugging:

Agent A: Trace the failure from input to output.
Agent B: Start from the observed symptom and generate competing hypotheses.
Agent C: Inspect recent changes and likely regression points.

For planning:

Agent A: Minimal-change plan.
Agent B: Robust long-term plan.
Agent C: Risk-first plan.

They are still doing the same job, but exploring different regions of the solution space.

You can also use:

- different temperatures;
- different context ordering;

- separate tool sessions;
- independent worktrees;
- different models at the same price tier.

9. The commands

The user-facing command set could remain compact:

```
/sdlc run  
/sdlc status  
/sdlc stage <stage>  
/sdlc retry  
/sdlc ensemble [count]  
/sdlc escalate  
/sdlc verify  
/sdlc finish
```

Examples:

```
/sdlc run "Fix issue #123"  
  
/sdlc ensemble 3 plan  
  
/sdlc escalate implement  
  
/sdlc verify
```

Internally, the orchestrator translates those commands into sub-agent assignments.

10. The first version I would actually build

Do not implement all six stages immediately.

Start with:

```
/explore  
/plan  
/verify
```

Use the main LLM session as the orchestrator and give it the ability to:

1. launch one cheap agent;
2. launch several cheap agents in parallel;
3. send results to a judge;

4. retry with feedback;
5. call one stronger model;
6. retain the selected output.

Then test it on ten different tasks:

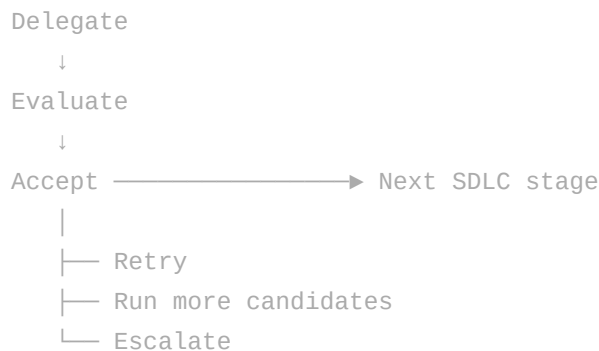
- three bugs;
- three features;
- two refactors;
- one investigation;
- one documentation task.

=

Only after seeing where it fails should you add implementation permissions, persistent state, or more sophisticated orchestration.



The core loop is small:



That, I think, is the real foundation of your idea: **an SDLC where intelligence is allocated dynamically rather than assigning the most expensive model to every task from the beginning.**

... Surse